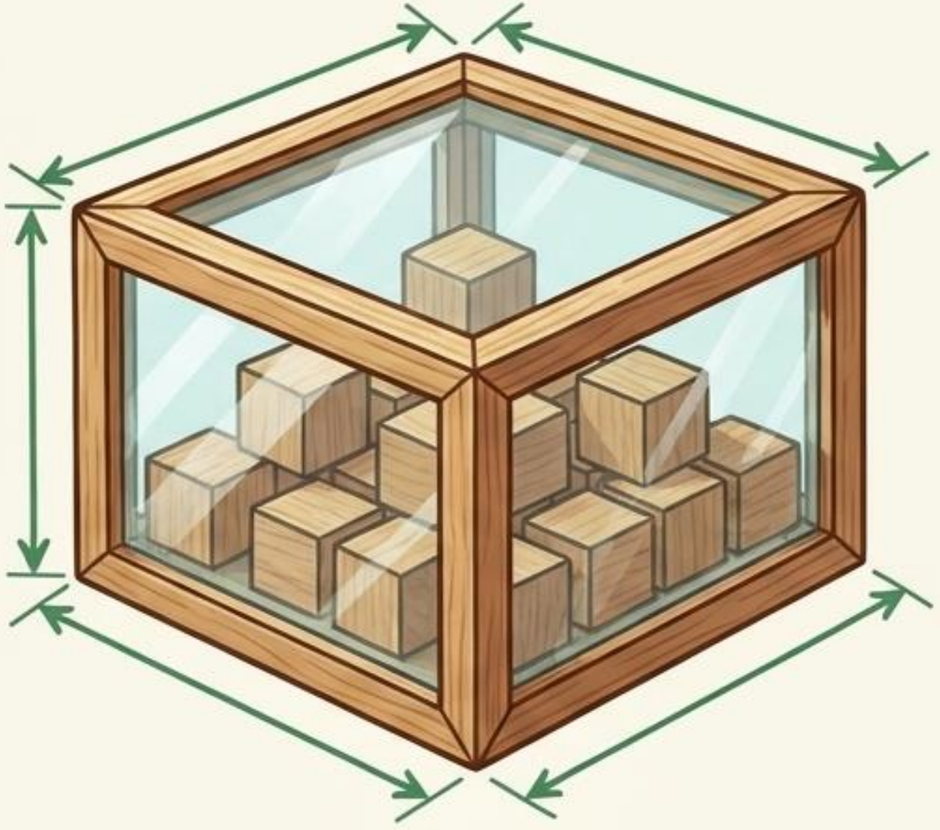
CH3 Python 程式碼的收納藝術

從單一檔案到系統化架構：模組、套件與獨立程式完全指南



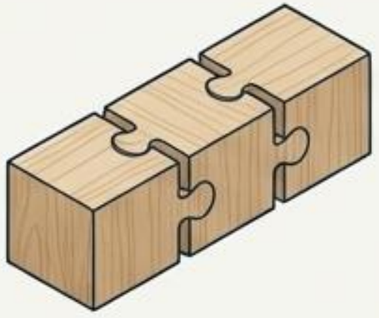
NotebookLM



Stage 1

單字 (拼圖)

關鍵字、變數與運算子
(if, for, while)



Stage 2

句子 (積木)

多個單字組合成的
一行程式



Stage 3

段落 (文件區塊)

多行程式組合成的函式
(Function)



Stage 4

文章 (完整書頁)

多個函式組合成的模組
(Module)

> **模組，其實就是一個 .py 檔案。它讓程式碼可以不斷被其他程式再利用。**

NotebookLM

打造第一個專屬模組：猜拳遊戲

```
import random

status = ['剪刀', '石頭', '布']

def figure_guess():
    return random.choice(status)
```

匯入內建隨機模組

定義包含三種拳型的串列

封裝功能，隨機回傳其中一個元素

NotebookLM

呼叫外部支援：匯入整個模組

將主程式與 guess.py 放在同一個資料夾下即可直接連線。

```
import guess
computer = guess.figure_guess()
print(computer)
```

必須使用『模組名稱.函式名稱()』來執行

```
> python main.py
石頭
```

NotebookLM

精準匯入與化名：讓程式碼更簡潔

只拿需要的工具

```
from guess import figure_guess
```

之後呼叫時直接寫 figure_guess()，不需再加模組前綴。

為模組取個綽號

```
import guess as gs
```

呼叫時簡化為 gs.figure_guess()。適合名稱過長的模組。

NotebookLM

尋寶地圖：Python 去哪裡找模組？

```
import sys
for path in sys.path:
    print(path)
```

系統會優先找尋『執行程式所在的資料夾』，若找到就不會往下一站找尋。

1. 當前資料夾 → 2. 內建函式庫 → 3. 第三方套件庫

```
K:\mybook\python\ch7
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib\site-packages
```

NotebookLM

當模組變多時：升級為「套件 (Packages)」

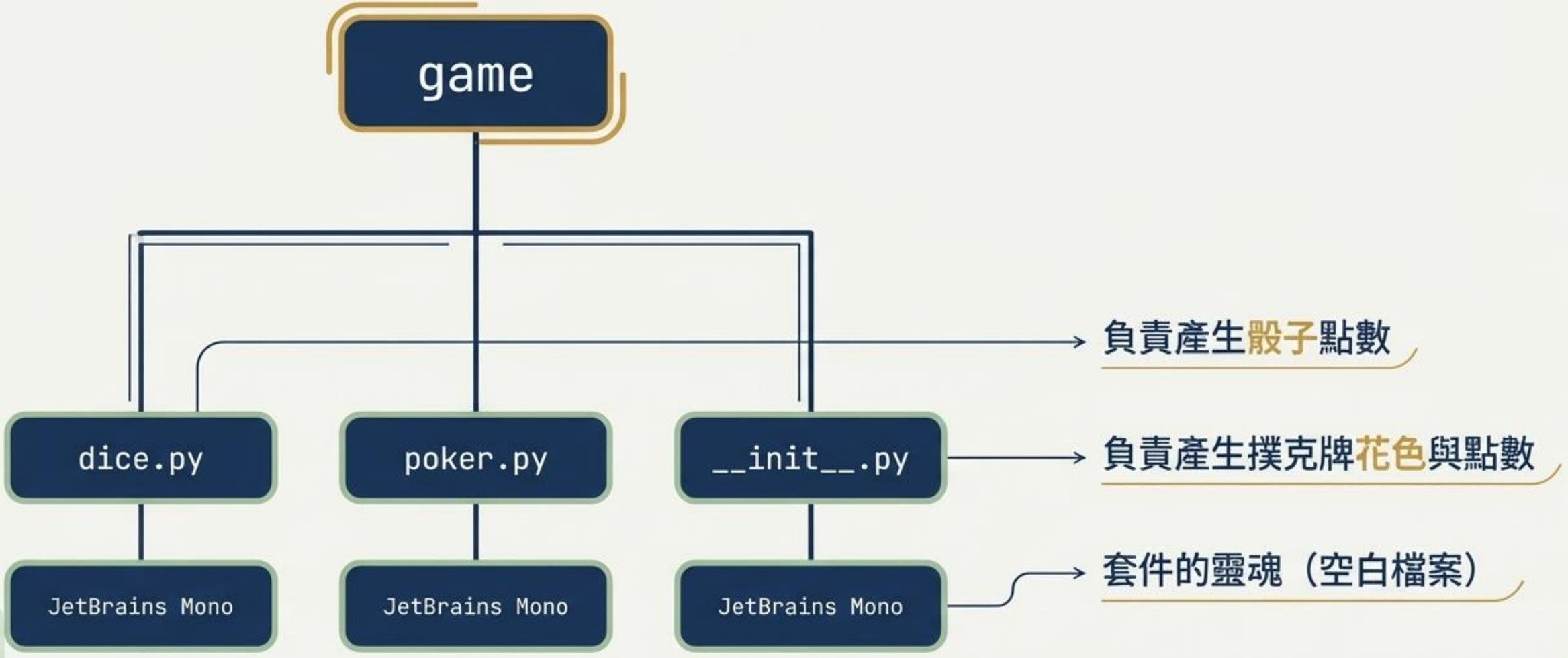


多個模組放在同一個資料夾下，並新增一個名為 **__init__.py** 的檔案，該資料夾就正式成為**套件**。

** 註：這個檔案可以是空的，它的存在就是一種「身分證明」。*

NotebookLM

實戰演練：建構遊戲套件結構



NotebookLM

透視套件內部：骰子與撲克牌

dice.py

```
from random import choice

def dice():
    return choice(range(1,7,1))
```

隨機回傳 1 到 6

poker.py

```
from random import choice

def poker():
    a = ['C', 'H', 'D', 'S']
    b = [1, 2, 3, 4, 5, 6, 7, 8, 9, 'T', 'J', 'Q', 'K']
    return choice(a)+str(choice(b))
```

隨機挑選花色與點數並字串串接

NotebookLM

呼叫武器庫：從套件匯入模組

from 套件名稱 import 模組名稱1, 模組名稱2

```
from game import dice, poker

for i in range(2):
    print(dice.dice())
    print(poker.poker())
```



透過套件，我們能有條理地管理並呼叫不同類型的遊戲功能。

NotebookLM

檔案的雙重身分：是工具還是主角？

身分一：被匯入的模組



當被其他程式 import 時，默默提供內部函式，不主動執行多餘動作。

```
if __name__ == '__main__':
```

身分二：獨立執行的腳本



當直接點擊或執行該檔案時，能自動啟動專屬的任務流程。

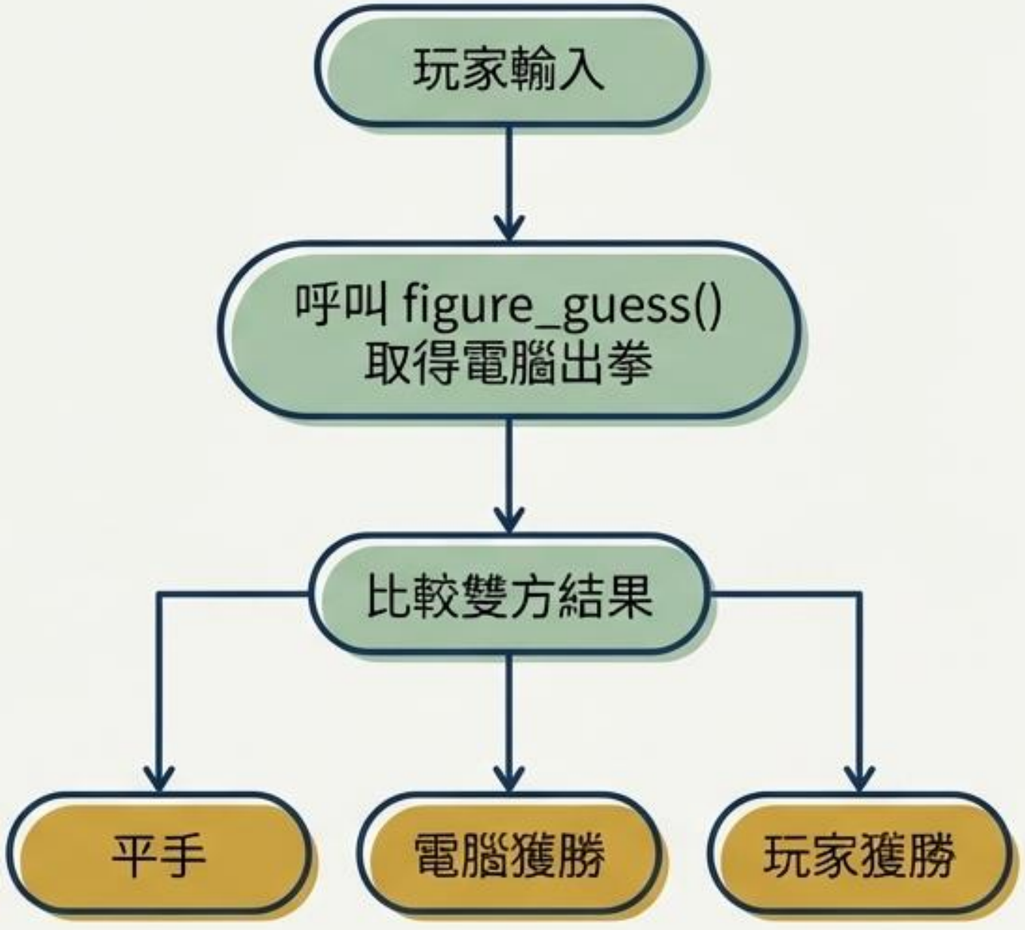
NotebookLM

實作大會串：完整的猜拳遊戲 (guess2.py)

```
from guess import figure_guess

def run():
    computer = figure_guess()
    my = input('請輸入剪刀、石頭或布？')

    # ... 比較邏輯（平手/電腦獲勝/玩家獲勝）...
```



NotebookLM

啟動魔法條件：守護獨立執行的防線

```
if __name__ == '__main__':
    for i in range(10):
        run()

else:
    print('我不是獨立執行的 python 程式')
```

當檔案被直接執行時，連續玩 10 局猜拳。

當檔案被當作模組匯入時，遊戲不會自動開始，而是印出提示訊息。

NotebookLM

驗證時刻：執行結果對比

直接執行 guess2.py

```
請輸入「剪刀」、「石頭」或「布」？剪刀
電腦出 布
玩家獲勝
```

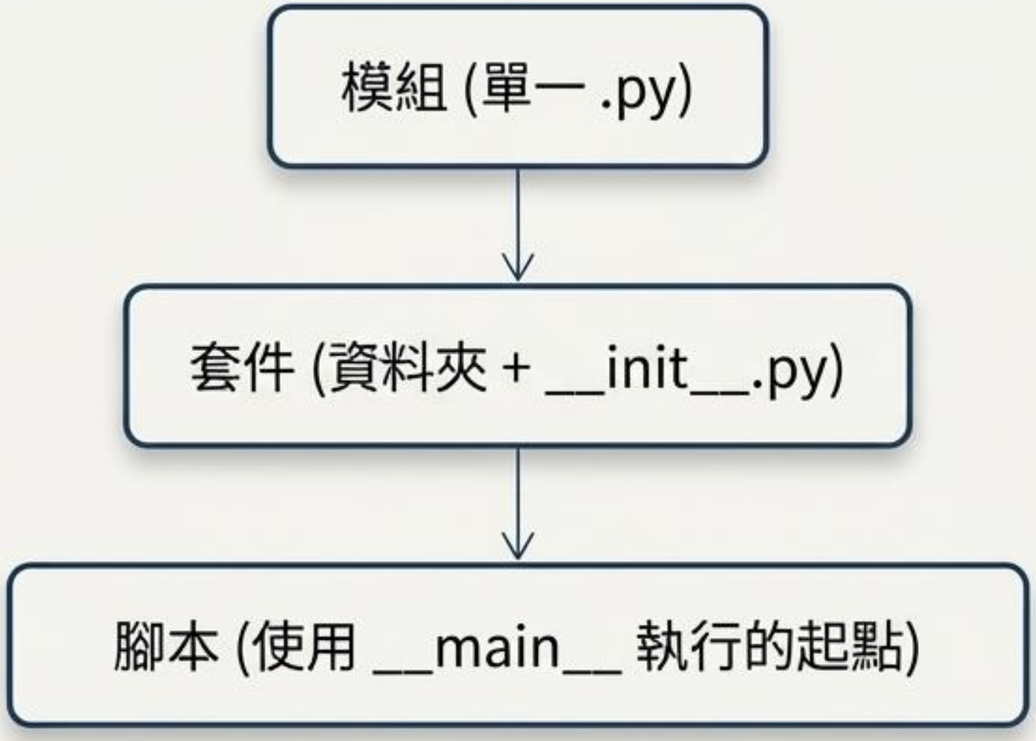
在其他檔案中匯入 import guess2

```
我不是獨立執行的 python 程式
```

守門員機制完美發揮作用，確保模組安全且腳本可被順利觸發。

NotebookLM

核心回顧：Python 組織架構全景圖



匯入語法速查清單

✓ `import module`	匯入全部
✓ `from module import func`	精準匯入
• `import module as md`	取別名匯入
• `from package import module`	套件匯入

> 優秀的程式碼不僅在於邏輯正確，更在於優雅的收納。
> 掌握模組與套件，就是掌握擴展系統的鑰匙。

NotebookLM